

MPM Cloth Simulator

The Big Picture

The project's end goal (Week 12) is a **hybrid cloth simulator** — a neural network predicts most simulation steps cheaply, and a physics fallback catches the hard frames. This architecture is taken directly from the seed paper [paper7]: *"We primarily leverage neural physics for fast updates, but incorporate a safeguard mechanism to fall back to MPM in challenging scenarios"*. Before you can train a neural network, you need:

1. A **reference physics simulator** to generate ground-truth trajectories.
2. A **fallback solver** to use when the neural network fails.
3. A **dataset** of trajectories to train on.
4. An **evaluation harness** to measure accuracy and speed.

That is exactly what M2 delivers. Everything in this repo is the foundation the neural and hybrid layers (M3–M5) will be built on.

The MPM Cloth Simulator Code Highlights

(src/mpm_cloth.py)

1 What is MPM?

The Material Point Method (MPM) is a hybrid simulation technique [paper1]. Instead of choosing between particles (which handle large deformations well but make contact hard) or meshes (which make forces easy but tangle under large deformation), MPM uses **both at once**:

- **Particles** carry the physical state: position $\mathbf{x}$, velocity $\mathbf{v}$, and a 3×3 *deformation gradient* $\mathbf{F}$ that records how much the material has stretched or sheared.
- A **background grid** is used temporarily each timestep to compute forces and resolve contact. The grid is then discarded.

Our simulator uses the **Moving Least Squares MPM (MLS-MPM)** variant [paper2]. The key insight of MLS-MPM is that the same affine matrix $\mathbf{C}_p$ (originally introduced by **Affine Particle-In-Cell** [APIC]) that carries momentum in the P2G/G2P transfers is *also* the

MLS-reconstructed velocity gradient $\partial \mathbf{v} / \partial \mathbf{x}$. This means $\mathbf{F}$ can be updated without any explicit kernel-gradient evaluation — a significant simplification over traditional MPM [paper2, §3.4].

Every timestep is three phases — P2G, grid update, G2P:

Particles —P2G→ Grid —update→ Grid —G2P→ Particles

(carry state) (compute forces) (receive new velocity)

The neural network in M3 will learn to *replace the grid update phase* with a cheap prediction. That is why the grid-update output — the per-particle acceleration $\mathbf{a}$ — is the most important thing saved in every trajectory clip [paper7, paper9, paper10].

2 The 64×64 Cloth Sheet

The cloth is represented as a flat grid of **4096 particles** (64 rows × 64 columns), spaced evenly on a 1 m × 1 m square. The particle index is $i * 64 + j$ for row i , column j .

```
# _DerivedConsts in _derive_constants()
```

```
n_particles = 64 * 64 = 4096
```

```
n_elements = 2 * 63 * 63 = 7938 # two triangles per quad cell
```

Each quad cell is split into two triangles. The triangles are used to compute the membrane strain energy — how much the cloth has stretched from its rest shape.

This particle count (4096) is directly comparable to the datasets used in [paper7] (4k particles for Water 2D / Sand 2D) and [paper9] (1.9k–20k particles across domains), confirming that our setup is within the regime where learned simulators have been shown to work well.

3 Phase 1 — P2G (Particle to Grid)

The **p2g** kernel scatters each particle's mass and momentum to the 27 nearest grid nodes (a 3×3×3 cube, using quadratic B-spline weights with $M_p = 1 / (4 \Delta x^2)$) [paper2, §3.3.5]. This is the MLS-MPM variant: it also scatters an *affine matrix* C_p that encodes local velocity gradient, making the transfer more accurate than standard APIC [paper2, §3.4].

```
grid_v[base + offset] += weight * (  
    p_mass * v[p] + affine @ dpos + dt * fext
```

)

Three contributions per node:

- `p_mass * v[p]` — particle momentum
- `affine @ dpos` — affine correction (APIC; makes MLS-MPM stable and accurate [paper2, §3.4])
- `dt * fext` — elastic force from the cloth material, contributed pre-emptively

The last term (`dt * fext`) is where the cloth stiffness enters. `fext = -x.grad[p]` is the gradient of the elastic energy with respect to particle position, computed by Taichi's autodiff [paper12].

4 The Elastic Energy and Autodiff

Before P2G, the code computes the total elastic energy of the cloth using Taichi's automatic differentiation [paper12]:

```
self.total_energy[None] = 0.0
```

```
with ti.ad.Tape(self.total_energy):
```

```
    self._compute_total_energy()
```

```
# x.grad[p] is now  $\partial \text{total\_energy} / \partial x[p]$  for every particle p
```

`compute_total_energy` loops over all 7938 triangles and evaluates the **linear co-rotational strain energy** per triangle:

$$\psi = \mu \|F_{\text{mem}} - R\|^2 + \frac{1}{2} \lambda (J - 1)^2$$

Where:

- `F_mem` is the 2×2 membrane deformation gradient (how the triangle has stretched in-plane, computed by comparing current edge vectors to rest-pose edge vectors).
- `R` is the rotation part of `F_mem` (from polar decomposition via SVD).
- `F_mem - R` is the *symmetric strain* — stretching and shearing without rotation.
- `J = det(F_mem)` measures area change.
- μ and λ are the Lamé parameters derived from Young's modulus (10 kPa) and Poisson ratio (0.3).

This material model is the "linear co-rotational" constitutive law, which appears in [paper2]'s Table 3 as the base elastic case and is the same family used in AnisoMPM [paper4] before the anisotropic extension. The co-rotational term $\| \mathbf{F} - \mathbf{R} \|^2$ penalises stretching and shearing; the volume term $(J-1)^2$ penalises area change.

Because Taichi traces through `compute_total_energy` with its autodiff system [paper12], `x.grad[p]` gives the elastic force on particle `p` automatically — no manual derivation of force formulas needed.

5 Phase 2 — Grid Update

The `grid_update` kernel:

1. **Normalises** grid momentum by grid mass $\rightarrow$ grid velocity.
2. **Adds gravity**: `grid_v[I] += dt * [0, -9.81, 0]`. This is the symplectic Euler grid momentum update from [paper2, Eq. 26]: $(m \hat{v})^{n+1} = (m v)^n + \Delta t (m g + f)$.
3. **Sphere collision**: if a grid node is inside the sphere, projects out the normal velocity component so the cloth can't penetrate it. This is the grid-based contact/boundary projection (`Proj(v, n, B, μ)`) described in [paper2, Eq. 25].
4. **Box boundary conditions**: clamps velocity to zero at the domain walls (3-cell margin on all 6 faces).

```
if rel.norm() < sphere_r:
```

```
    n = rel.normalized()
```

```
    vn = grid_v[I].dot(n)
```

```
    if vn < 0.0:
```

```
        grid_v[I] -= vn * n # remove inward component; keep tangential
```

This is the step the neural network will replace in M3. The neural prediction receives the pre-grid particle state $(\mathbf{x}, \mathbf{v}, \mathbf{F})$ and outputs the per-particle acceleration that the grid update *would have produced* [paper7].

6 Phase 3 — G2P (Grid to Particle)

The `g2p` kernel is the mirror of P2G. Each particle interpolates its new velocity from the surrounding 27 grid nodes using the same quadratic B-spline weights.

It then updates:

- $a[p] = (new_v - v[p]) / dt$ — the per-particle acceleration (the M3 training label; same output convention as [\[paper7\]](#), [\[paper9\]](#), [\[paper10\]](#)).
- $v[p] = new_v$ — velocity from the grid.
- $x[p] += dt * v[p]$ — symplectic Euler position update.
- $C[p] = new_C$ — new affine matrix (APIC; used in the next P2G).
- $F[p] = (I + dt * C) @ F[p]$ — deformation gradient update [\[paper2, Eq. 17\]](#): $F^{n+1} = (I + \Delta t C_p^{n+1}) F^n$. This reuses the APIC C_p from G2P and requires no separate computation of ∇v from kernel gradients.
- `contact_flag[p]` — whether this particle is near the sphere surface.

7 Configuration (`configs/mpm.yaml`)

Every physical parameter is in one YAML file, making ablations easy:

Parameter	Value	Source
<code>cloth.grid</code>	<code>[64, 64]</code>	Matches particle counts in [paper7] (4k particles)
<code>mpm.grid_resolution</code>	128	128^3 background grid; $dx = 2.0/128 \approx 15$ mm
<code>mpm.dt_s</code>	<code>1e-4</code>	Within [paper2] 's range $\Delta t \in [5 \times 10^{-6}, 3 \times 10^{-4}]$ s
<code>material.young_modulus_*_pa</code>	<code>1e4</code>	Soft cloth; CFL-stable at this dt
<code>material.poisson_ratio</code>	<code>0.3</code>	Mild incompressibility (standard for cloth)
<code>contact.sphere.radius_m</code>	<code>0.2</code>	20 cm sphere; visible drape shape

The `dx_m` and `inv_dx` fields are computed automatically if set to `null`.

8 Public API

```
sim = MPMClothSim("configs/mpm.yaml")
```

```
sim.reset()          # place cloth flat above sphere; zero grid
```

```
sim.step()           # one full P2G → grid update → G2P cycle
```

```
state = sim.state()   # {"x", "v", "a", "F", "contact_flag", "step"}
```

```
traj = sim.rollout(7000) # 7000 steps, logs every step by default
```

`state()` returns the particle state dictionary whose fields map directly to the training data schema to be used by learned simulation models [paper7, paper9, paper10]: positions `x`, velocities `v`, deformation gradients `F`, and accelerations `a`. [paper2, §7] specifically recommends exposing `{x_p, v_p, F_p, C_p, m_p, σ_p}` as the training-data state for any neural grid-update replacement.